INFORME TÉCNICO - TAREA 5: CONSULTAS DML Y VISTAS ANALÍTICAS

PROYECTO: Red Social Estudiantil Pascualina

CURSO: Bases de Datos I

ESTUDIANTE: Weymar Lozano Mosquera

Institución Universitaria Pascual Bravo

FECHA: Noviembre 2026

Introducción

El presente informe documenta el diseño, la implementación y la validación del sistema de consultas DML y vistas analíticas sobre el modelo de datos relacional e híbrido de la Red Social Estudiantil Pascualina.

El objetivo principal es transformar los datos almacenados en información estratégica de negocio mediante sentencias SQL optimizadas, garantizando el cumplimiento de reglas de nomenclatura (uso de ALIAS), integridad referencial y explotación de datos semiestructurados (IoT/JSON).

Estructura del Modelo Físico

El modelo relacional físico consta de 14 tablas consolidadas e interconectadas mediante claves primarias (PK) y claves foráneas (FK) con restricciones de borrado en cascada (ON DELETE CASCADE):

usuarios: Entidad principal de autenticación (id_usuario [PK], nombre_completo, correo_institucional, carrera_programa, estado_cuenta, fecha_registro).

perfiles_usuario: Relación 1:1 con usuarios (id_perfil [PK], id_usuario [FK/UNIQUE], biografia, semestre_actual, url_foto_perfil, intereses_academicos).

roles_sistema: Catálogo de roles de acceso (id_rol [PK], nombre_rol, descripcion_rol).

usuario_roles: Tabla intermedia N:M (id_usuario_rol [PK], id_usuario [FK], id_rol [FK]).

conexiones_seguidores: Red de contactos (id_conexion [PK], id_usuario_seguidor [FK], id_usuario_seguido [FK], tipo_vinculo, fecha_conexion).

publicaciones: Muro de contenidos (id_publicacion [PK], id_usuario_autor [FK], categoria_post, contenido_texto, fecha_publicacion).

multimedia_publicacion: Adjuntos multimedia (id_multimedia [PK], id_publicacion [FK], tipo_archivo, url_recurso).

comentarios: Respuestas a publicaciones (id_comentario [PK], id_publicacion [FK], id_usuario [FK], texto_comentario, fecha_comentario).

reacciones: Valoraciones sociales (id_reaccion [PK], id_publicacion [FK], id_usuario [FK], tipo_reaccion).

grupos: Comunidades de estudio (id_grupo [PK], id_usuario_creador [FK], nombre_grupo, descripcion, categoria_grupo).

miembros_grupo: Nómina de grupos N:M (id_miembro_grupo [PK], id_grupo [FK], id_usuario [FK], rol_en_grupo, fecha_ingreso).

eventos: Agenda académica (id_evento [PK], id_usuario_organizador [FK], id_grupo [FK], titulo_evento, ubicacion_o_enlace, fecha_inicio).

asistentes_evento: Participación en eventos (id_evento [FK], id_usuario [FK]).

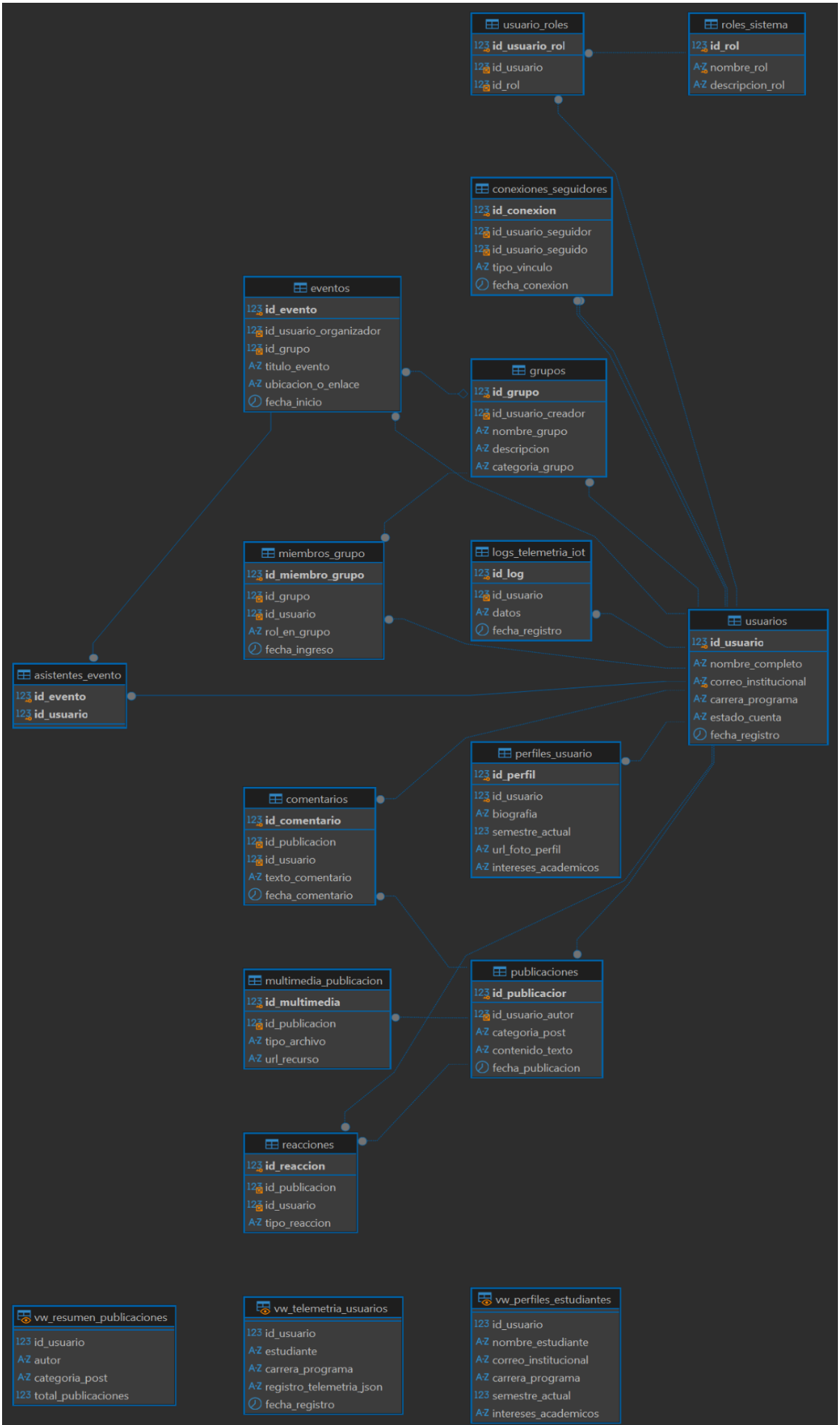
logs_telemetria_iot: Telemetría y campus inteligente (id_log [PK], id_usuario [FK], datos [JSON/TEXT], fecha_registro).

EXPLICACIÓN DEL TRABAJO

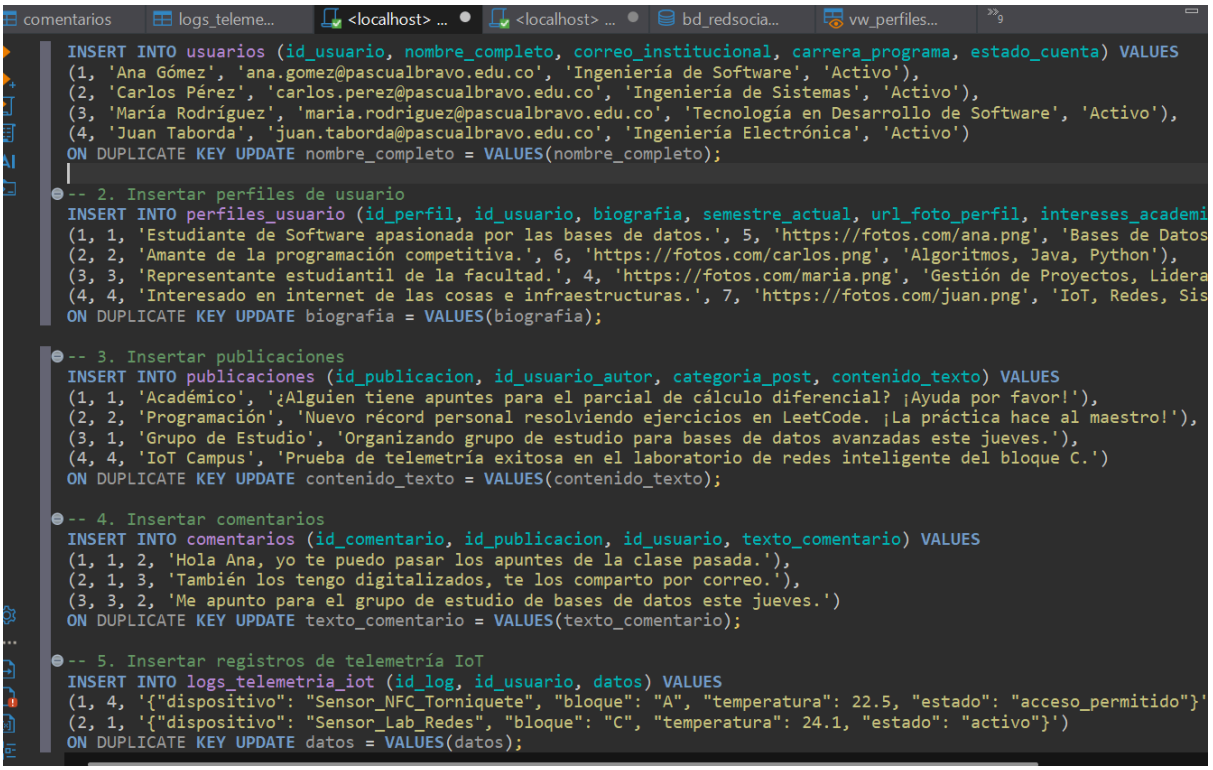
La estrategia metodológica adoptada para el desarrollo de la Tarea 5 se dividió en tres fases colaborativas:

1. Fase de brainstorming y análisis de negocio: El equipo definió las preguntas de negocio clave requeridas para evaluar el comportamiento de los estudiantes en la plataforma.
2. Fase de depuración de consultas DML y vistas: Se construyeron los scripts DML iterativamente. Cada consulta fue probada en los SGBD seleccionados, refinando el uso de JOINs, subconsultas y cláusulas de agregación para evitar redundancias.
3. Diseño y validación del script DML final, implementación de subconsultas avanzadas (NOT IN, EXISTS), optimización de vistas analíticas y redacción general del informe técnico.

Diagrama de modelo de base de datos


usuario_roles
id_usuario_rol
id_usuario
id_rol
roles_sistema
id_rol
nombre_rol
descripcion_rol
conexiones_seguidores
id_conexion
id_usuario_seguidor
id_usuario_seguido
tipo_vinculo
fecha_conexion
eventos
id_evento
id_usuario_organizador
id_grupo
titulo_evento
ubicacion_o_enlace
fecha_inicio
grupos
id_grupo
id_usuario_creador
nombre_grupo
descripcion
categoria_grupo
miembros_grupo
id_miembro_grupo
id_grupo
id_usuario
rol_en_grupo
fecha_ingreso
logs_telemetria_iot
id_log
id_usuario
datos
fecha_registro
usuarios
id_usuario
nombre_completo
correo_institucional
carrera_programa
estado_cuenta
fecha_registro
asistentes_evento
id_evento
id_usuario
perfiles_usuario
id_perfil
id_usuario
biografia
semestre_actual
url_foto_perfil
intereses_academicos
comentarios
id_comentario
id_publicacion
id_usuario
texto_comentario
fecha_comentario
multimedia_publicacion
id_multimedia
id_publicacion
tipo_archivo
url_recurso
publicaciones
id_publicacior
id_usuario_autor
categoria_post
contenido_texto
fecha_publicacion
reacciones
id_reaccion
id_publicacion
id_usuario
tipo_reaccion
vw_resumen_publicaciones
id_usuario
autor
categoria_post
total_publicaciones
vw_telemetria_usuarios
id_usuario
estudiante
carrera_programa
registro_telemetria_json
fecha_registro
vw_perfiles_estudiantes
id_usuario
nombre_estudiante
correo_institucional
carrera_programa
semestre_actual
intereses_academicos

Inserciones en tablas



SISTEMA DE CONSULTAS DML SIGNIFICATIVAS

Consulta 1: Publicaciones recientes con autores y comentarios (Cruce de 3+ Tablas)

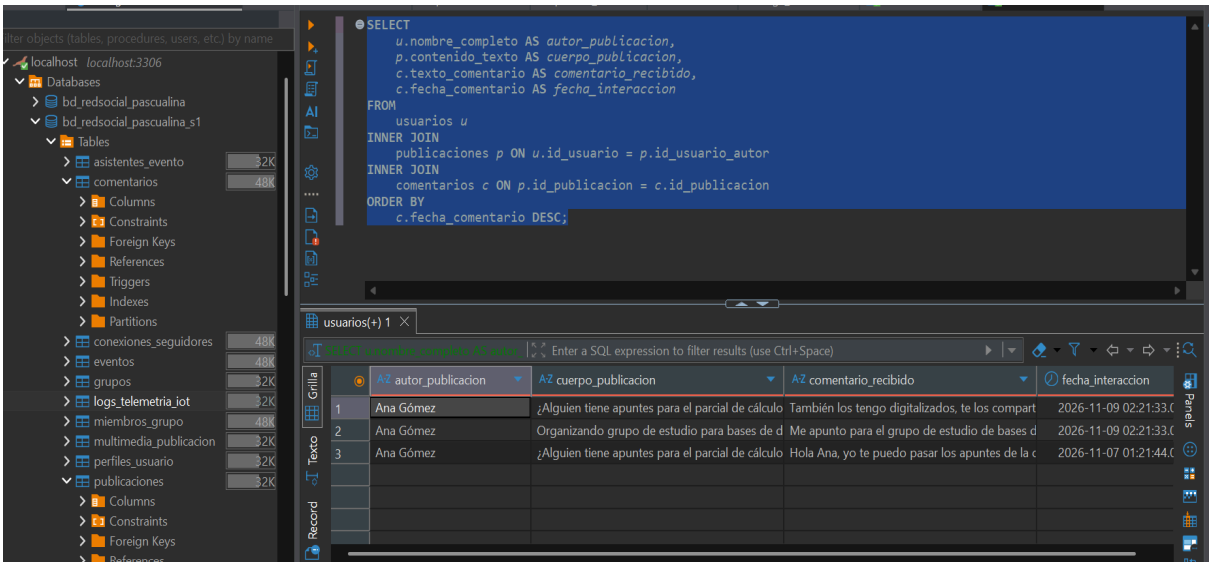
- **Pregunta de negocio:** ¿Cuáles son las publicaciones recientes en la plataforma, qué estudiante las redactó y qué comentarios han recibido?
- **Sentencia SQL:**

```
SELECT
  u.nombre_completo AS autor_publicacion,
  p.contenido_texto AS cuerpo_publicacion,
  c.texto_comentario AS comentario_recibido,
  c.fecha_comentario AS fecha_interaccion
FROM
  usuarios u
INNER JOIN
  publicaciones p ON u.id_usuario = p.id_usuario_autor
INNER JOIN
  comentarios c ON p.id_publicacion = c.id_publicacion
ORDER BY
  c.fecha_comentario DESC;
```

- **Explicación detallada:** Combina tres tablas relacionales (usuarios, publicaciones y comentarios) utilizando la sintaxis INNER JOIN. Muestra la interacción completa del feed y ordena los resultados del comentario más reciente al más antiguo.
- **Justificación de ALIAS:** Se asignaron alias de tabla (u, p, c) para abreviar las referencias y alias de columna (AS autor_publicacion, AS cuerpo_publicacion, etc.) para dar claridad técnica y semántica al reporte generado.

Consulta 2: Identificación de usuarios inactivos (consulta de diferencia)

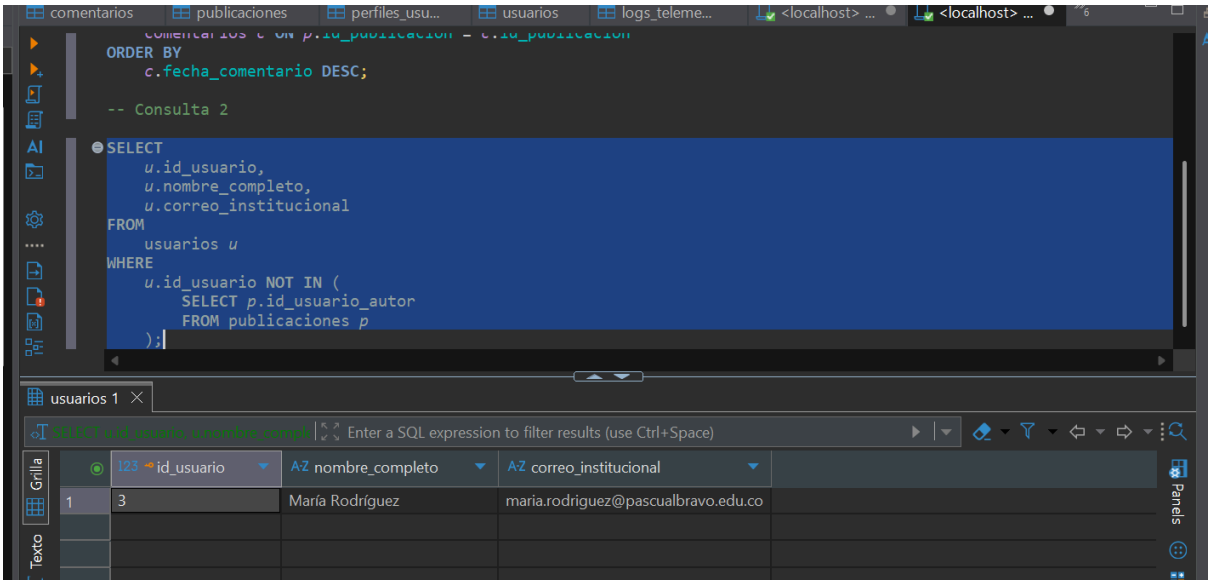
- **Pregunta de negocio:** ¿Qué estudiantes registrados en la red social no han



realizado ninguna publicación en el muro?

- **Sentencia SQL:**

```sql
SELECT
  u.id_usuario,
  u.nombre_completo,
  u.correo_institucional
FROM
  usuarios u
WHERE
  u.id_usuario NOT IN (
    SELECT p.id_usuario_autor
    FROM publicaciones p
  );
```

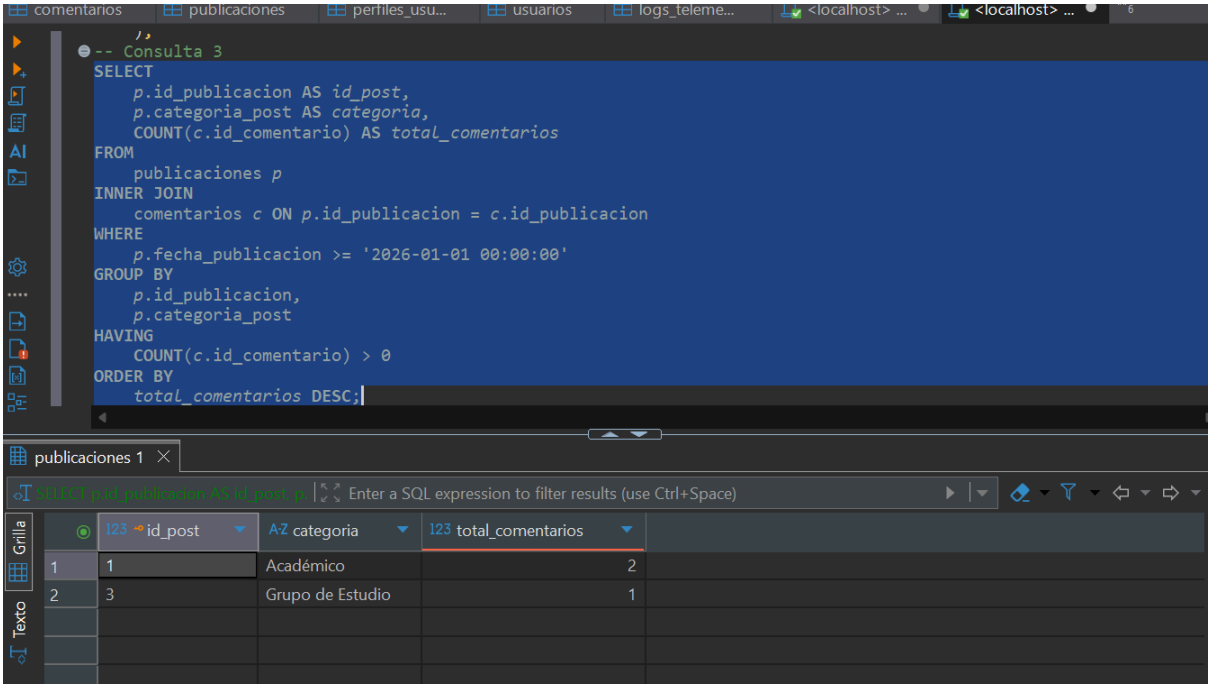
- **Explicación detallada:** Aplica una lógica de diferencia de conjuntos mediante el operador NOT IN. La subconsulta extrae la lista de id_usuario_autor que han publicado, y la consulta externa extrae a los estudiantes que no pertenecen a esa lista.
- **Justificación de ALIAS:** Los alias AS id_estudiante, AS nombre_estudiante y AS email_institucional estandarizan la salida para reportes administrativos de usuarios pasivos.
- 

Consulta 3: Ranking de publicaciones populares (uso de todas las cláusulas SQL)

- **Pregunta de negocio:** ¿Cuántos comentarios acumulan las publicaciones realizadas en el año actual, filtrando solo aquellas que tengan al menos 1 comentario y ordenadas de mayor a menor popularidad?
- **Sentencia SQL:**

```
SELECT
  p.id_publicacion AS id_post,
  p.categoria_post AS categoria,
  COUNT(c.id_comentario) AS total_comentarios
FROM
  publicaciones p
INNER JOIN
  comentarios c ON p.id_publicacion = c.id_publicacion
WHERE
  p.fecha_publicacion >= '2026-01-01 00:00:00'
GROUP BY
  p.id_publicacion,
  p.categoria_post
```

```
HAVING
  COUNT(c.id_comentario) > 0
ORDER BY
  total_comentarios DESC;
```

- **Explicación detallada:** Implementa las 6 cláusulas fundamentales del SQL (SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY). Filtra los datos base por fecha, agrupa por publicación, evalúa condiciones sobre la función agregada COUNT() mediante HAVING y ordena los resultados cuantitativamente.
- **Justificación de ALIAS:** AS id_post y AS total_comentarios identifican unívocamente los valores calculados facilitando la lectura analítica.
- 

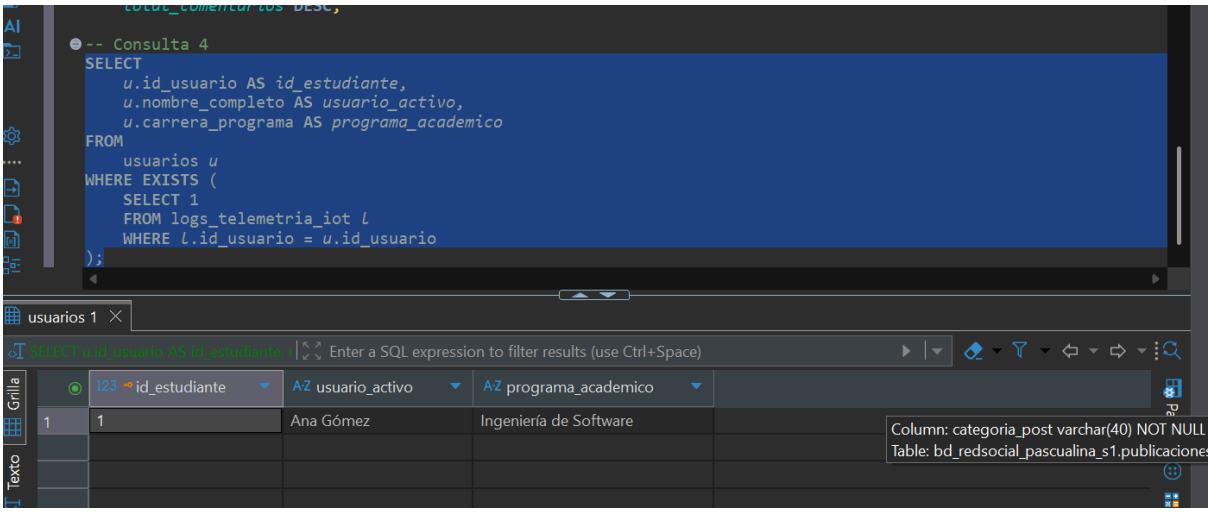
Consulta 4: Usuarios con registros de telemetría IoT (subconsulta correlacionada EXISTS)

- **Pregunta de negocio:** ¿Qué usuarios poseen interacciones o registros de acceso capturados por los sensores IoT del campus inteligente?
- **Sentencia SQL:**

```
SELECT
  u.id_usuario AS id_estudiante,
  u.nombre_completo AS usuario_activo,
  u.carrera_programa AS programa_academico
FROM
  usuarios u
WHERE EXISTS (
  SELECT 1
```

```
  FROM logs_telemetria_iot l
  WHERE l.id_usuario = u.id_usuario
);
```

- **Explicación detallada:** Emplea una subconsulta correlacionada con EXISTS que evalúa la presencia de registros en la tabla logs_telemetria_iot sin duplicar ni aplanar las filas del usuario.
- **Justificación de ALIAS:** Alias de tabla (u, l) simplifican la correlación en la cláusula WHERE.

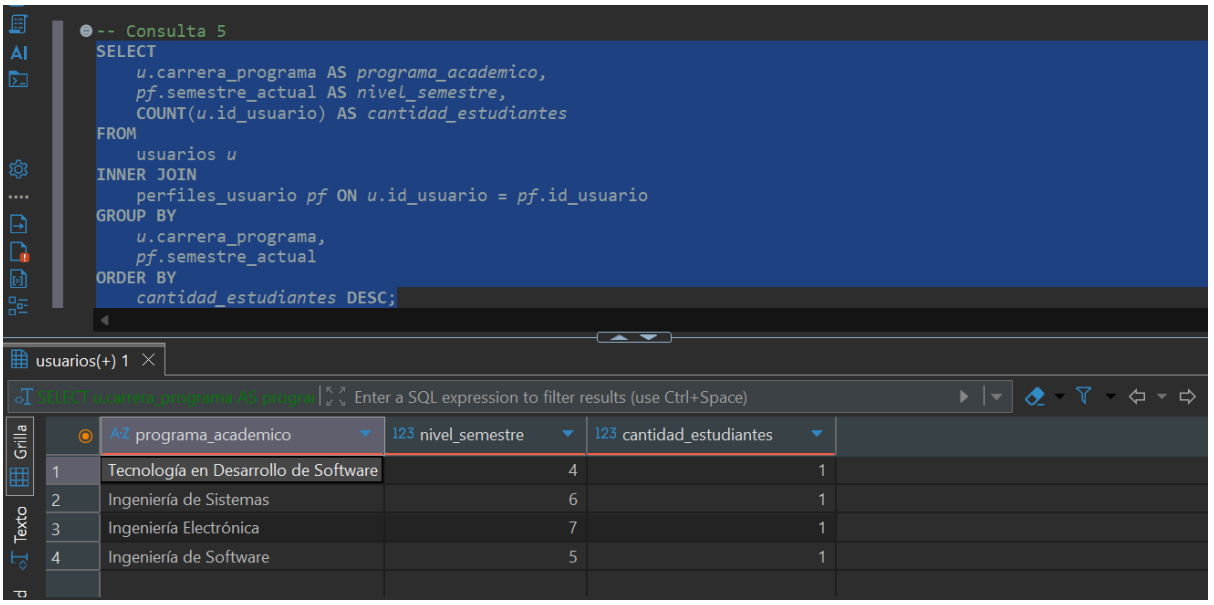


Consulta 5: Distribución demográfica de estudiantes por programa y semestre

- **Pregunta de negocio:** ¿Cuántos estudiantes están registrados en la plataforma agrupados por su programa académico y semestre actual?
- **Sentencia SQL:**

```
SELECT
  u.carrera_programa AS programa_academico,
  pf.semestre_actual AS nivel_semestre,
  COUNT(u.id_usuario) AS cantidad_estudiantes
FROM
  usuarios u
INNER JOIN
  perfiles_usuario pf ON u.id_usuario = pf.id_usuario
GROUP BY
  u.carrera_programa,
  pf.semestre_actual
ORDER BY
  cantidad_estudiantes DESC;
```

- **Explicación detallada:** Agrupa los registros combinados entre usuarios y perfiles_usuario para generar una métrica cuantitativa agrupada multidimensionalmente.
- **Justificación de ALIAS:** Se transforman nombres de campos técnicos en descriptores formales (AS programa_academico, AS nivel_semestre, AS cantidad_estudiantes).



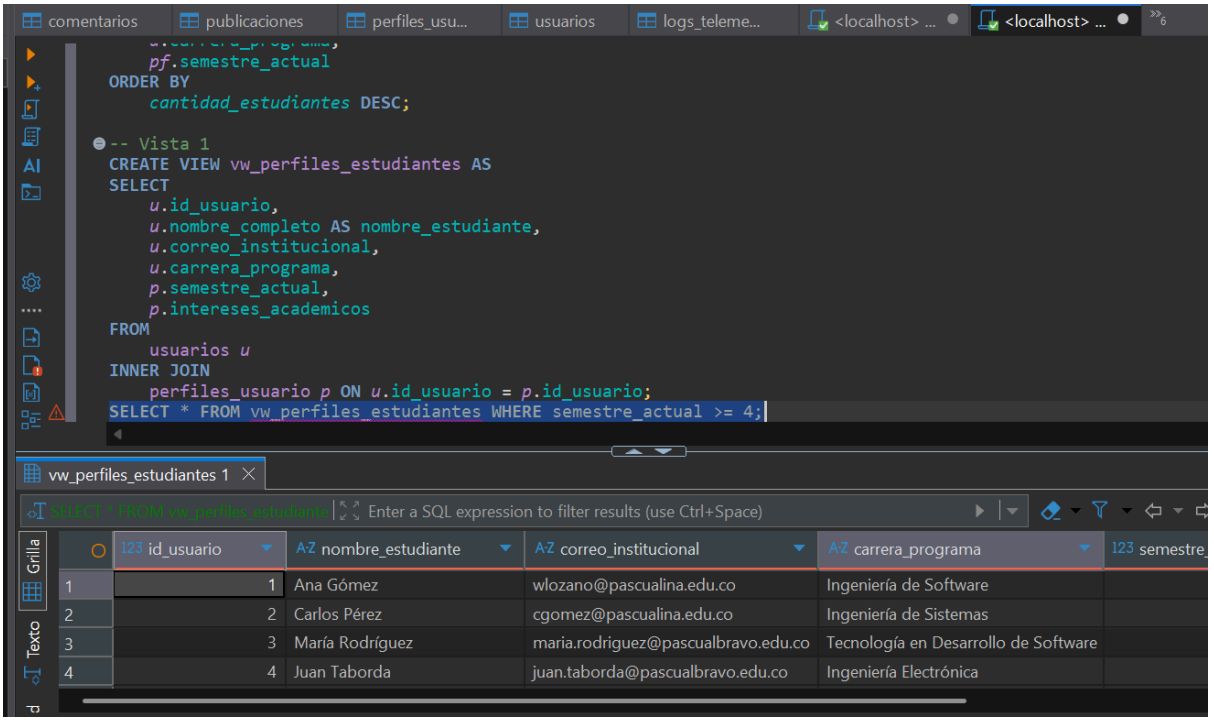
MODELO DE CONSULTAS ALMACENADAS EN VISTAS ANALÍTICAS (MÍNIMO 3)

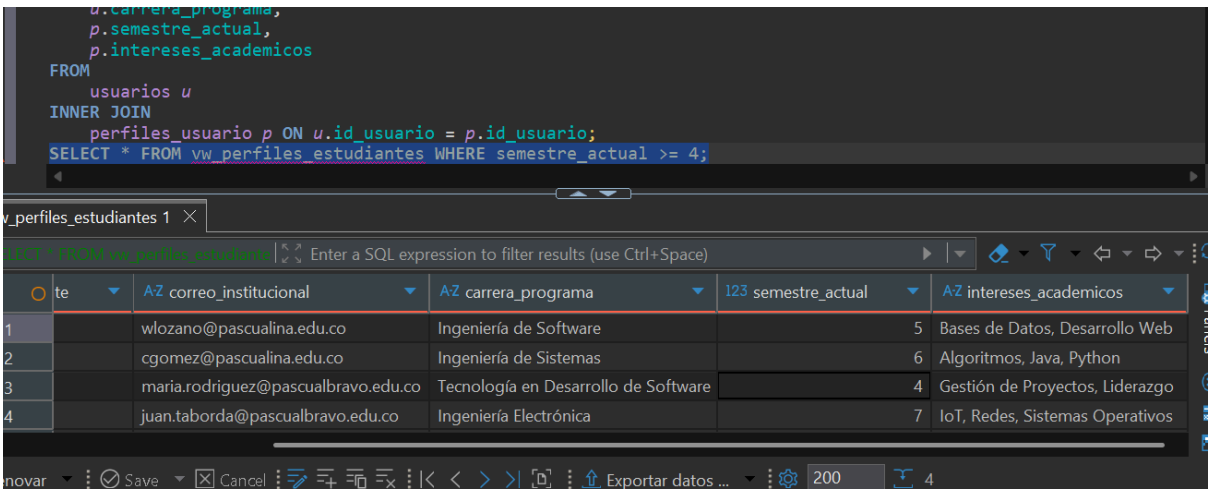
Vista 1: Consolidación de perfiles e intereses estudiantiles (vw_perfiles_estudiantes)

- **Objetivo de la vista:** Consolidar en una vista lógica los datos personales, académicos e intereses de los estudiantes para agilizar la búsqueda de mentores o compañeros de estudio.
- **Sentencia SQL de creación:**

```
CREATE VIEW vw_perfiles_estudiantes AS
SELECT
  u.id_usuario,
  u.nombre_completo AS nombre_estudiante,
  u.correo_institucional,
  u.carrera_programa,
  p.semestre_actual,
  p.intereses_academicos
FROM
  usuarios u
INNER JOIN
  perfiles_usuario p ON u.id_usuario = p.id_usuario;
SELECT * FROM vw_perfiles_estudiantes WHERE semestre_actual >= 4;
```

- **Justificación del potencial analítico: E**vita la redacción repetitiva de INNER JOIN entre la tabla de identidades y perfiles. Permite que aplicaciones web o sistemas de Business Intelligence filtren directamente por carreras e intereses académicos.



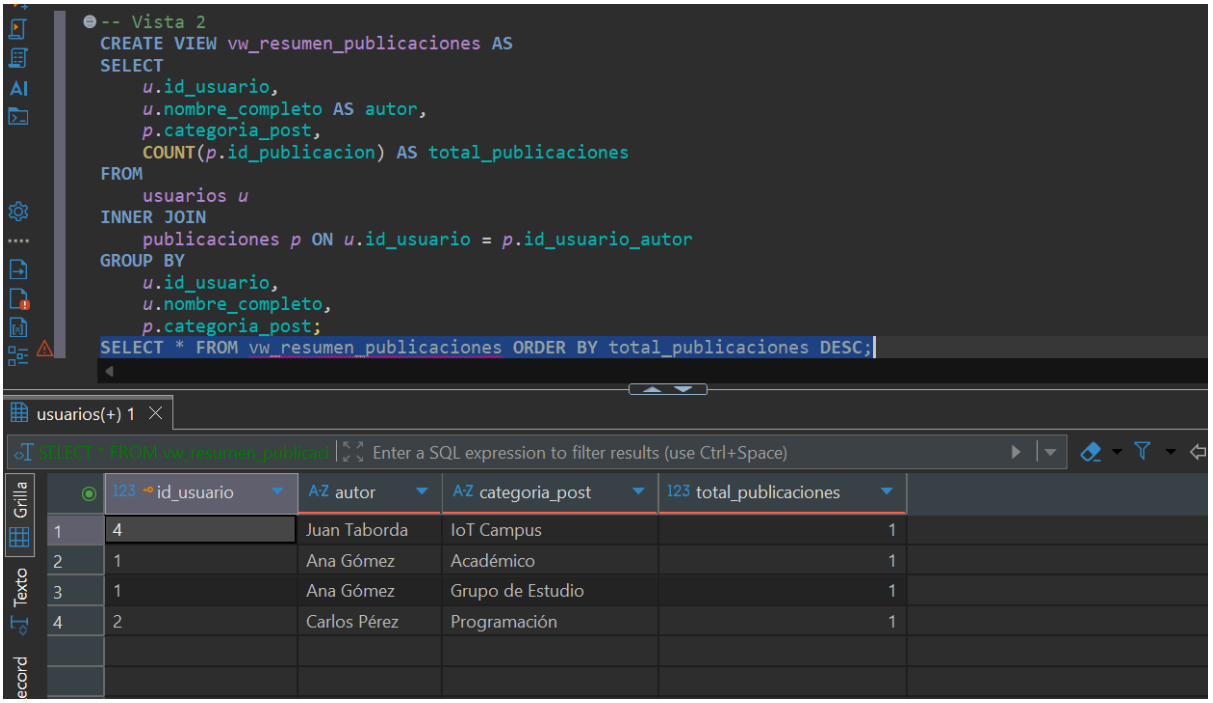


Vista 2: Resumen de actividad de publicaciones por autor (vw_resumen_publicaciones)

- **Objetivo de la vista:** Cuantificar el volumen total de publicaciones generadas por cada estudiante agrupadas por categoría temática.
- **Sentencia SQL de creación:**

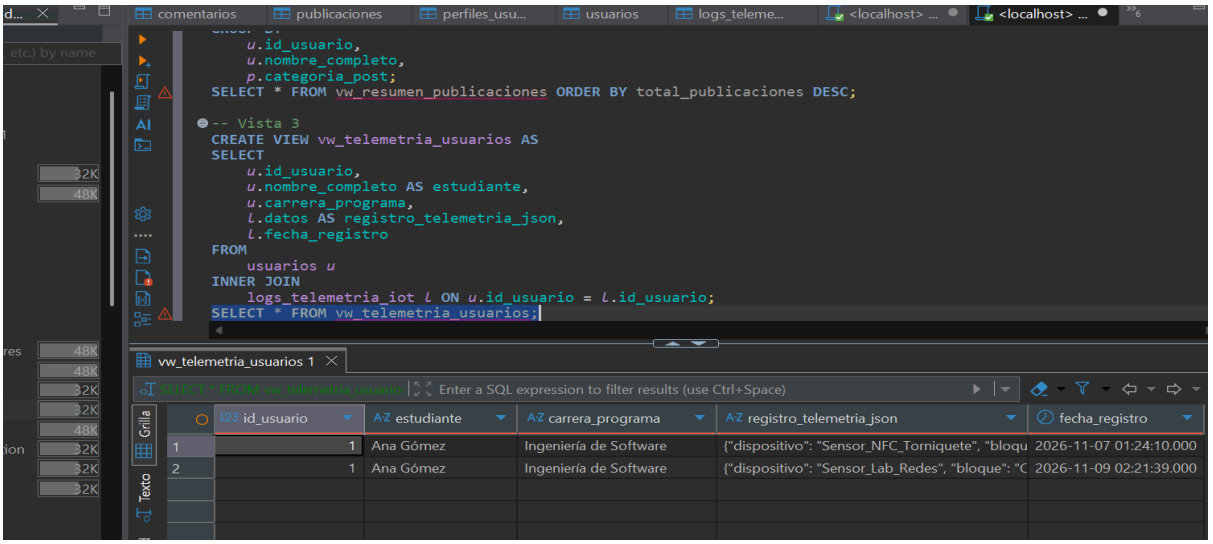
```sql
-- Vista 2
CREATE VIEW vw_resumen_publicaciones AS
SELECT
  u.id_usuario,
  u.nombre_completo AS autor,
  p.categoria_post,
  COUNT(p.id_publicacion) AS total_publicaciones
FROM
  usuarios u
INNER JOIN
  publicaciones p ON u.id_usuario = p.id_usuario_autor
GROUP BY
  u.id_usuario,
  u.nombre_completo,
  p.categoria_post;
SELECT * FROM vw_resumen_publicaciones ORDER BY total_publicaciones DESC;
```

- **Justificación del potencial analítico:** Simplifica las métricas de engagement y participación estudiantil. Facilita la generación de dashboards sobre qué áreas del conocimiento generan mayor interacción.



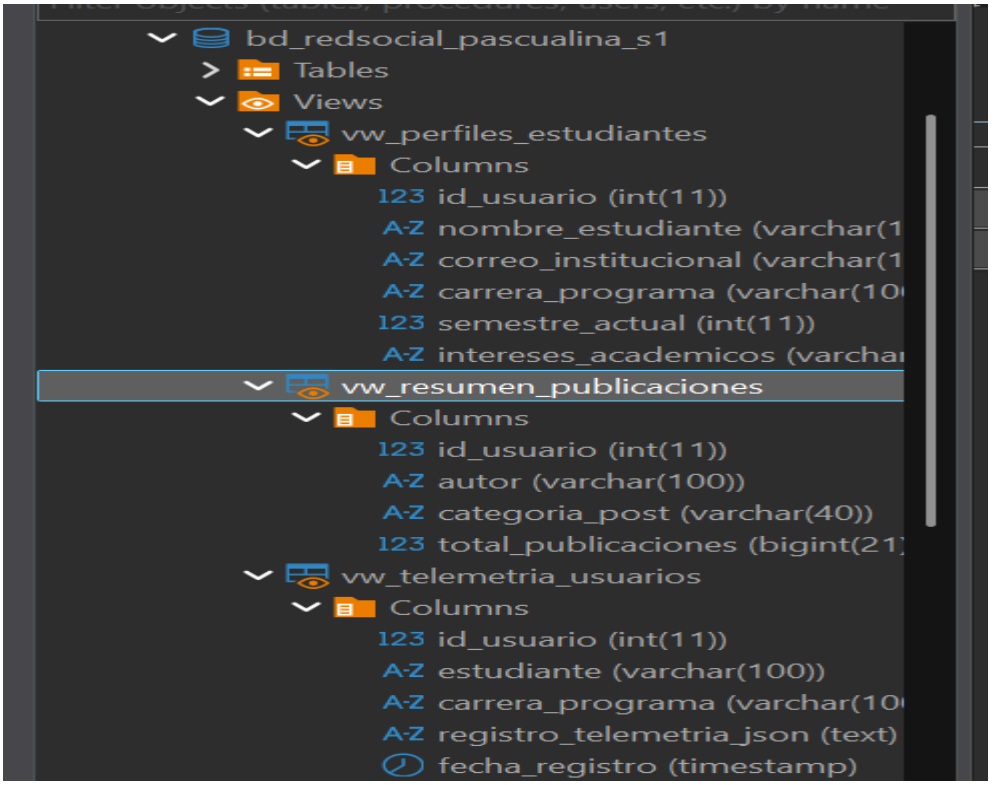
Vista 3: Auditoría y telemetría IoT de usuarios (vw_telemetria_usuarios)

- **Objetivo de la vista:** Integrar los registros de telemetría IoT del campus inteligente con los datos formales del usuario.
- **Sentencia SQL de creación:**



- **Justificación del potencial analítico:** Combina estructuras relacionales rígidas con datos semiestructurados (JSON), permitiendo auditar el uso de laboratorios y accesos al campus por programa académico.

Total de vistas



DIFICULTADES, VENTAJAS Y DESVENTAJAS

Dificultades

1. **Nomenclatura estricta de columnas:** Se presentaron errores tipo Unknown column durante las ejecuciones iniciales debido a discrepancias entre los nombres de los atributos relacionales (id_usuario_autor vs id_usuario, contenido_texto vs contenido). Esto requirió una verificación directa sobre el diccionario del motor.
2. **Validación de tablas vacías:** Al ejecutar subconsultas como NOT IN, los resultados devolvían conjuntos vacíos. Se resolvió insertando usuarios de control sin publicaciones asociadas.

Ventajas Observadas

- **Abstracción con vistas:** Simplifican radicalmente la complejidad de las sentencias SQL para los desarrolladores frontend.
- **Uso de ALIAS:** Aumentan exponencialmente la legibilidad del código y la presentación formal de los reportes.

Desventajas Observadas

- **Rendimiento de subconsultas correlacionadas:** Subconsultas con EXISTS sobre volúmenes masivos de datos pueden ralentizar el motor si los campos clave no están adecuadamente indexados.

CONCLUSIÓN INDIVIDUAL

El desarrollo de esta actividad me permitió afianzar el dominio práctico del lenguaje DML y la creación de vistas analíticas en SQL. Comprendí que la correcta aplicación de cláusulas de agregación (GROUP BY, HAVING) y la implementación de subconsultas lógicas no solo resuelven preguntas complejas de negocio, sino que optimizan el procesamiento de datos en plataformas estudiantiles de alto tráfico.